

Guía de estudio

Reconstrucción de un proyecto Spring Boot con Maven

Investiga cada pieza, relaciónala con el ciclo de vida del proyecto y úsala para reconstruir su árbol mínimo sin mirar el IDE.

META DE LA GUÍA

Al terminar, podrás cerrar el IDE y dibujar de memoria un proyecto mínimo de Spring Boot con Maven. Junto a cada elemento anotarás: quién lo lee, cuándo se usa y si contiene código fuente o un resultado generado.

Cómo estudiar esta guía

- No memorices solo nombres de carpetas. Pregunta qué herramienta necesita cada elemento y en qué momento del trabajo lo consulta.
- Para cada tema, busca una fuente confiable, abre un proyecto real y contrasta la teoría con lo que ves en el explorador de archivos.
- Responde las preguntas con tus propias palabras. Si una respuesta depende de otra pieza, dibuja una flecha entre ambas.
- Evita copiar el árbol final de esta guía durante la autoevaluación. La práctica final está diseñada para revelar qué relaciones todavía no puedes explicar.

Leyenda de uso

Etiqueta	Qué debes poder decidir
Fuente	Lo escribe y mantiene una persona; participa al compilar, probar o ejecutar.
Generado	Lo crea una herramienta como Maven; normalmente puede borrarse y regenerarse.
Lector	La herramienta o proceso que interpreta el archivo o directorio: Maven, Maven Wrapper, compilador Java, Spring Boot, pruebas o IDE.

Ruta recomendada: primero entiende Maven y su Wrapper; después distingue el código de aplicación, los recursos y las pruebas; por último identifica qué produce la compilación.

1. Descriptor de Maven

Ubicación o nombre:

pom.xml

QUÉ ES

El Project Object Model, o POM, es el archivo XML que describe el proyecto para Maven. Declara su identidad, versiones, dependencias, complementos de compilación y otras decisiones del proceso de construcción.

PARA QUÉ EXISTE

Existe para que Maven pueda construir el proyecto de forma repetible. En vez de instalar bibliotecas o ejecutar pasos manualmente, Maven lee reglas declaradas en el POM y resuelve lo necesario.

Qué debes investigar

- Qué datos forman las coordenadas de un artefacto: groupId, artifactId y version.
- Cómo se declaran dependencias y qué significa su scope, especialmente compile, runtime y test.
- Para qué sirven parent, properties, build y plugins en un proyecto Spring Boot.
- Qué ocurre cuando Maven resuelve dependencias desde repositorios y las guarda localmente.

Conceptos relacionados que debes poder explicar

- modelo de proyecto
- dependencia transitiva
- BOM y gestión de versiones
- plugin de Maven
- ciclo de vida: validate, compile, test, package
- artefacto JAR

Errores o confusiones comunes

- Confundir el POM con un archivo que ejecuta la aplicación. Maven lo lee; Spring Boot no lo usa para arrancar en producción.
- Creer que todas las dependencias se copian dentro del repositorio. Maven las descarga desde repositorios configurados.
- Editar versiones al azar sin entender el parent o la gestión de dependencias de Spring Boot.

Preguntas de comprobación

1. ¿Qué lector principal interpreta pom.xml y en qué acciones lo consulta?
2. ¿Qué diferencia hay entre una dependencia y un plugin?
3. ¿Por qué el scope test evita que una biblioteca de pruebas sea parte de la aplicación final?
4. ¿Qué información permite a Maven distinguir tu proyecto de otro artefacto?

PISTA PARA LA RECONSTRUCCIÓN

Es un archivo fuente de configuración del proceso de construcción. Dibújalo en la raíz del proyecto y une una flecha hacia Maven.

2. Maven Wrapper

Ubicación o nombre:

mvnw, mvnw.cmd y .mvn/wrapper

QUÉ ES

El Maven Wrapper es un conjunto de scripts y archivos de configuración que permite ejecutar una versión concreta de Maven para el proyecto, incluso si Maven no está instalado globalmente en el equipo.

PARA QUÉ EXISTE

Existe para reducir diferencias entre máquinas y automatizaciones. El equipo puede usar el comando incluido por el proyecto y obtener la versión de Maven indicada por el Wrapper.

Qué debes investigar

- Qué ejecutan ./mvnw en sistemas tipo Unix y mvnw.cmd en Windows.
- Qué guarda .mvn/wrapper/maven-wrapper.properties y por qué apunta a una distribución de Maven.
- La diferencia entre Maven instalado globalmente y Maven obtenido por el Wrapper.
- Cómo el Wrapper termina invocando Maven, que luego lee pom.xml.

Conceptos relacionados que debes poder explicar

- script de arranque
- portabilidad
- versionado de herramientas
- integración continua
- distribución de Maven
- permisos de ejecución

Errores o confusiones comunes

- Pensar que mvnw es el compilador Java. Es un iniciador que asegura Maven; el compilador se usa después dentro del proceso.
- Borrar .mvn porque parece una carpeta auxiliar. Puede ser necesaria para que el Wrapper sepa qué versión usar.
- Usar mvn y ./mvnw como si siempre fueran equivalentes; pueden seleccionar versiones distintas.

Preguntas de comprobación

1. ¿Qué problema del equipo resuelve el Maven Wrapper?
2. ¿Qué elemento del proyecto indica la versión de Maven que debe usarse?
3. ¿Qué cadena de lectura ocurre cuando escribes ./mvnw test?
4. ¿Por qué el Wrapper es útil en integración continua?

PISTA PARA LA RECONSTRUCCIÓN

Son scripts y configuración fuente del proyecto. Ubícalos en la raíz, junto a pom.xml; el Wrapper conduce a Maven, y Maven conduce al POM.

3. Carpeta principal de Java

Ubicación o nombre:

src/main/java

QUÉ ES

Es la ubicación convencional de Maven para el código Java de la aplicación. Allí viven los paquetes, clases, interfaces y otros tipos que se compilan para formar el programa principal.

PARA QUÉ EXISTE

Existe para separar el código que forma parte de la aplicación de los recursos y de las pruebas. Maven reconoce esta convención y la envía al compilador durante la fase compile.

Qué debes investigar

- Cómo se relacionan paquetes Java y directorios, por ejemplo com.ejemplo.demo con com/ejemplo/demo.
- Qué hace el compilador Java con archivos .java y dónde deja sus .class generados.
- La diferencia entre código de producción y código de prueba.
- Qué clases suelen aparecer en una aplicación web de Spring: controladores, servicios, repositorios y configuración.

Conceptos relacionados que debes poder explicar

- paquete
- clase
- compilación
- classpath
- bytecode
- convenciones de Maven
- código de producción

Errores o confusiones comunes

- Creer que el nombre de la carpeta src/main/java forma parte del package de Java. Es una raíz de fuentes, no un paquete.
- Guardar archivos .class junto a los .java. Los .class deben ser resultados generados, normalmente bajo target.
- Poner pruebas en main porque también son Java; eso cambia qué se incluye al empaquetar.

Preguntas de comprobación

1. ¿Qué herramienta compila los .java de esta carpeta y en qué fase?
2. ¿Por qué la ruta de un paquete debe coincidir con su declaración package?
3. ¿Es fuente o generado el contenido de src/main/java?
4. ¿Qué efecto tendría mover una clase de producción a src/test/java?

PISTA PARA LA RECONSTRUCCIÓN

Marca este directorio como fuente. Maven lo reconoce y el compilador Java transforma sus archivos en clases generadas para la aplicación.

4. Clase de arranque de Spring Boot

Ubicación o nombre:

src/main/java/.../NombreAplicacion.java

QUÉ ES

Es la clase Java que contiene el método main y suele estar anotada con `SpringBootApplication`. Es el punto habitual desde el cual se inicia la aplicación Spring Boot.

PARA QUÉ EXISTE

Existe para crear el contexto de Spring, aplicar la configuración automática y arrancar el tipo de aplicación elegido, por ejemplo una aplicación web con servidor embebido.

Qué debes investigar

- Qué hace el método main y cómo `SpringApplication.run` inicia el contexto.
- Qué reúne la anotación `SpringBootApplication`: configuración, configuración automática y exploración de componentes.
- Por qué conviene ubicar esta clase en un paquete raíz por encima de controladores y servicios.
- La diferencia entre ejecutar desde el IDE, con Maven y desde un JAR ya empaquetado.

Conceptos relacionados que debes poder explicar

- JVM
- método main
- contexto de aplicación
- component scan
- auto-configuración
- bean
- servidor embebido

Errores o confusiones comunes

- Pensar que Maven lee esta clase para decidir dependencias. Maven la compila y puede usarla al empaquetar, pero la JVM/Spring la ejecutan al arrancar.
- Confundir la clase principal con todas las configuraciones de la aplicación. Es el punto de inicio, no necesariamente el lugar de toda la lógica.
- Ubicarla demasiado abajo en la jerarquía de paquetes y dejar componentes fuera del escaneo.

Preguntas de comprobación

1. ¿Quién ejecuta el método main cuando la aplicación arranca?
2. ¿Qué responsabilidad tiene `SpringBootApplication`?
3. ¿Por qué importa el paquete de esta clase?
4. ¿Cuál es la diferencia entre compilar la clase y ejecutarla?

PISTA PARA LA RECONSTRUCCIÓN

Es código fuente dentro de `src/main/java`. En tu dibujo, destaca su método main y anota: la JVM inicia; Spring Boot construye el contexto.

5. Recursos

Ubicación o nombre:

src/main/resources

QUÉ ES

Es la ubicación convencional de archivos no Java necesarios para la aplicación: configuración, plantillas, archivos estáticos, mensajes y otros recursos que deben estar disponibles en el classpath.

PARA QUÉ EXISTE

Existe para que Maven copie esos archivos junto al resultado compilado sin tratarlos como código Java. Spring Boot puede leerlos durante el arranque o atenderlos mientras la aplicación se ejecuta.

Qué debes investigar

- Qué suele contener application.properties o application.yml y cómo Spring Boot lo usa para configurar la aplicación.
- Qué diferencia hay entre recursos de configuración, plantillas y contenido estático.
- A dónde Maven copia los recursos al construir y cómo terminan en el JAR.
- Qué significa classpath cuando una aplicación busca un recurso.

Conceptos relacionados que debes poder explicar

- configuración externa
- properties y YAML
- classpath
- perfil de Spring
- plantillas
- contenido estático
- empaquetado

Errores o confusiones comunes

- Llamar código fuente Java a un archivo properties o YAML. Es fuente de configuración, no código Java compilable.
- Editar application.properties dentro de target esperando conservar el cambio. Ese archivo suele ser una copia generada desde src/main/resources.
- Confundir recursos de la aplicación con dependencias de Maven.

Preguntas de comprobación

1. ¿Qué lector puede usar application.properties y en qué momento?
2. ¿Por qué Maven copia recursos aunque no los compile como Java?
3. ¿Qué implica que un recurso esté en el classpath?
4. ¿Qué cambio se perdería si editas solo la copia bajo target?

PISTA PARA LA RECONSTRUCCIÓN

Son fuentes de recursos. Maven las procesa o copia al construir; Spring Boot puede leer la configuración al inicio y usar otros recursos en ejecución.

6. Carpeta de pruebas

Ubicación o nombre:

Carpeta de pruebas: src/test

QUÉ ES

Es la zona convencional reservada para pruebas automáticas. En un proyecto Java suele incluir src/test/java para pruebas escritas en Java y src/test/resources para datos o configuración exclusiva de pruebas.

PARA QUÉ EXISTE

Existe para validar el comportamiento sin mezclar ese código con la aplicación que se entrega. Maven compila y ejecuta estas pruebas en una fase distinta del ciclo de vida.

Qué debes investigar

- La separación entre test source set y main source set.
- Qué hacen JUnit, Spring Boot Test y el plugin de pruebas de Maven.
- Cómo se usa src/test/resources para archivos disponibles durante las pruebas.
- Qué significa que una prueba pueda usar el código de main, mientras que main no debe depender del código de test.

Conceptos relacionados que debes poder explicar

- prueba unitaria
- prueba de integración
- JUnit
- test classpath
- Surefire
- cobertura
- aislamiento

Errores o confusiones comunes

- Creer que src/test nunca se compila. Sí se compila, pero para el ciclo de pruebas y con un classpath diferente.
- Poner una dependencia de producción con scope test y luego no encontrarla al ejecutar la aplicación.
- Interpretar una prueba que pasa en el IDE como garantía de que Maven test también pasará; pueden diferir configuración o perfil.

Preguntas de comprobación

1. ¿Cuándo se usa src/test dentro del ciclo de vida de Maven?
2. ¿Por qué una prueba puede importar clases de src/main/java?
3. ¿Qué no debería depender de src/test?
4. ¿Qué tipo de archivos ubicarías en src/test/resources?

PISTA PARA LA RECONSTRUCCIÓN

Trátalo como fuente de pruebas. Maven y su plugin de pruebas lo usan al ejecutar test; normalmente no forma parte del artefacto de producción.

7. Directorio de salida

Ubicación o nombre:

target

QUÉ ES

Es el directorio de salida convencional de Maven. Allí se reúnen los resultados de compilar, procesar recursos, ejecutar pruebas y empaquetar, como clases .class, informes y archivos JAR.

PARA QUÉ EXISTE

Existe para separar los productos del trabajo automatizado de los archivos que mantiene el equipo. Esto permite limpiar resultados antiguos y volver a construir de manera consistente.

Qué debes investigar

- Qué subdirectorios pueden aparecer: classes, test-classes, surefire-reports y archivos empaquetados.
- Qué hace mvn clean y por qué elimina target.
- Cómo un JAR empaquetado se relaciona con las clases y recursos generados o copiados.
- Por qué target suele estar en .gitignore.

Conceptos relacionados que debes poder explicar

- artefacto generado
- bytecode
- empaquetado
- informes de prueba
- clean
- reproducibilidad
- .gitignore

Errores o confusiones comunes

- Editar archivos bajo target para arreglar el proyecto. La siguiente construcción puede sobrescribirlos.
- Versionar target completo como si fuera código fuente. Normalmente se regenera desde pom.xml y src.
- Creer que target existe antes de compilar o que siempre tiene exactamente la misma estructura.

Preguntas de comprobación

1. ¿Quién crea target y en qué operaciones?
2. ¿Qué diferencia hay entre target/classes y src/main/java?
3. ¿Por qué es razonable borrar target antes de una construcción limpia?
4. ¿Qué evidencia de las pruebas puede encontrarse allí?

PISTA PARA LA RECONSTRUCCIÓN

Es generado. Dibuja target separado de src y anota que Maven lo crea durante compilación, pruebas y empaquetado; no es la fuente que se edita.

Autoevaluación y reconstrucción

Haz esta práctica con el IDE cerrado y sin consultar el árbol de un proyecto. No busques respuestas en esta guía mientras la realizas. El valor está en detectar qué relación todavía debes investigar.

Parte A Dibujo del árbol mínimo

En una hoja, dibuja desde la raíz un árbol mínimo de un proyecto Spring Boot con Maven. Debe incluir pom.xml, Maven Wrapper, src/main/java, la clase de arranque, src/main/resources, src/test y target. Puedes incluir los subdirectorios que consideres necesarios, pero no copies un árbol de ejemplo.

Elemento	Quién lo lee	Cuándo se usa	Fuente o generado
pom.xml			
Maven Wrapper			
src/main/java			
clase de arranque			
src/main/resources			
src/test			
target			

Parte B Explicación oral

- Señala una flecha desde el Wrapper hasta Maven y explica por qué ambos no son lo mismo.
- Señala una flecha desde Maven hacia pom.xml y describe qué decide Maven a partir de ese archivo.
- Señala src/main/java y target/classes, y explica qué transformación conecta ambos.
- Señala la clase de arranque y explica qué sucede después de su método main.
- Señala un recurso de configuración y explica por qué una copia dentro de target no debería ser tu fuente de edición.
- Señala src/test y explica qué puede consumir de main y qué no debe ir dentro del JAR de producción.

Parte C Criterio de salida

Considera la reconstrucción lograda cuando puedas dibujar el árbol, completar la tabla anterior y justificar cada decisión sin depender de frases memorizadas. Si dudas, vuelve al tema que conecte la herramienta, el momento de uso y el tipo de archivo que no pudiste explicar.